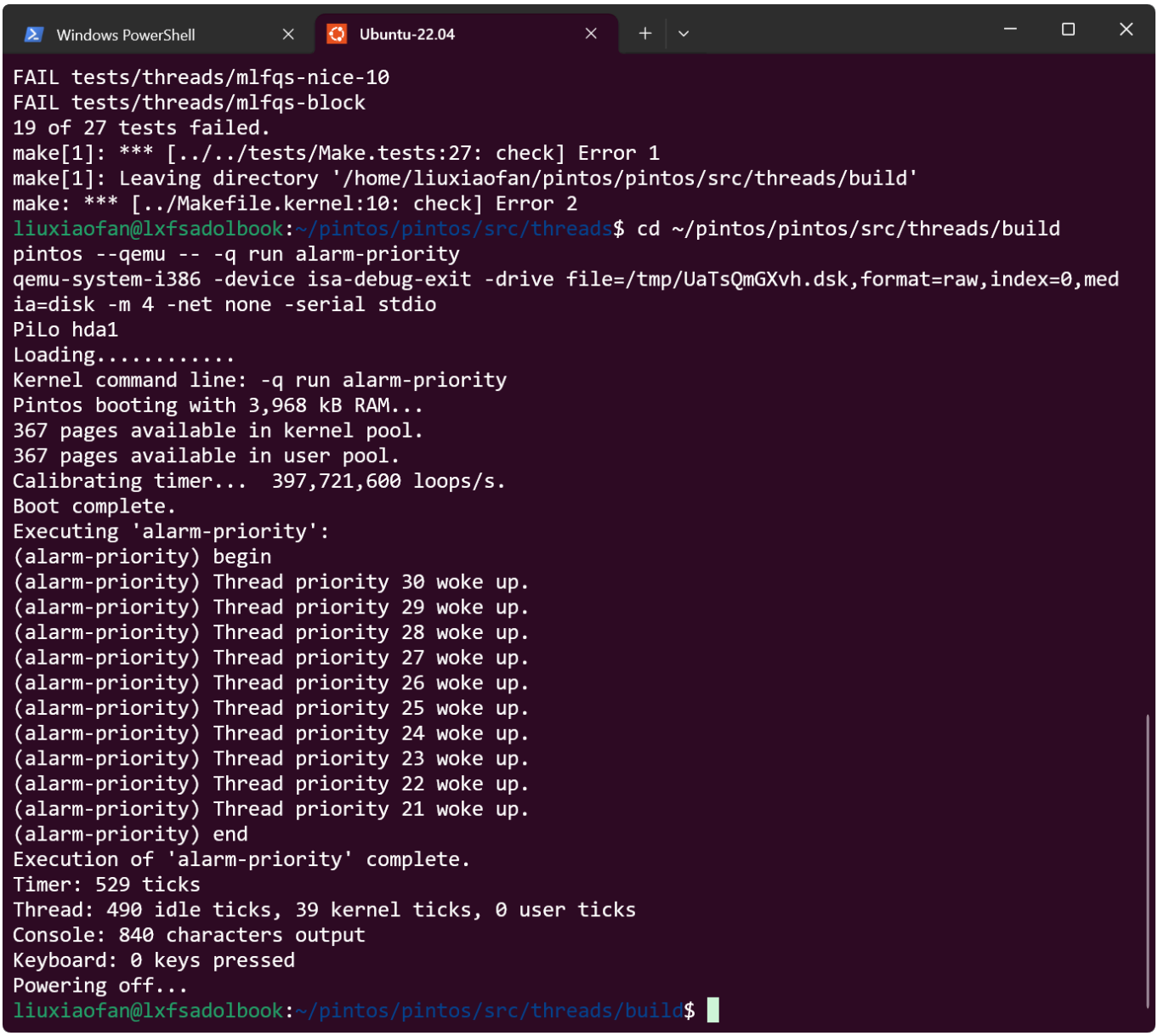
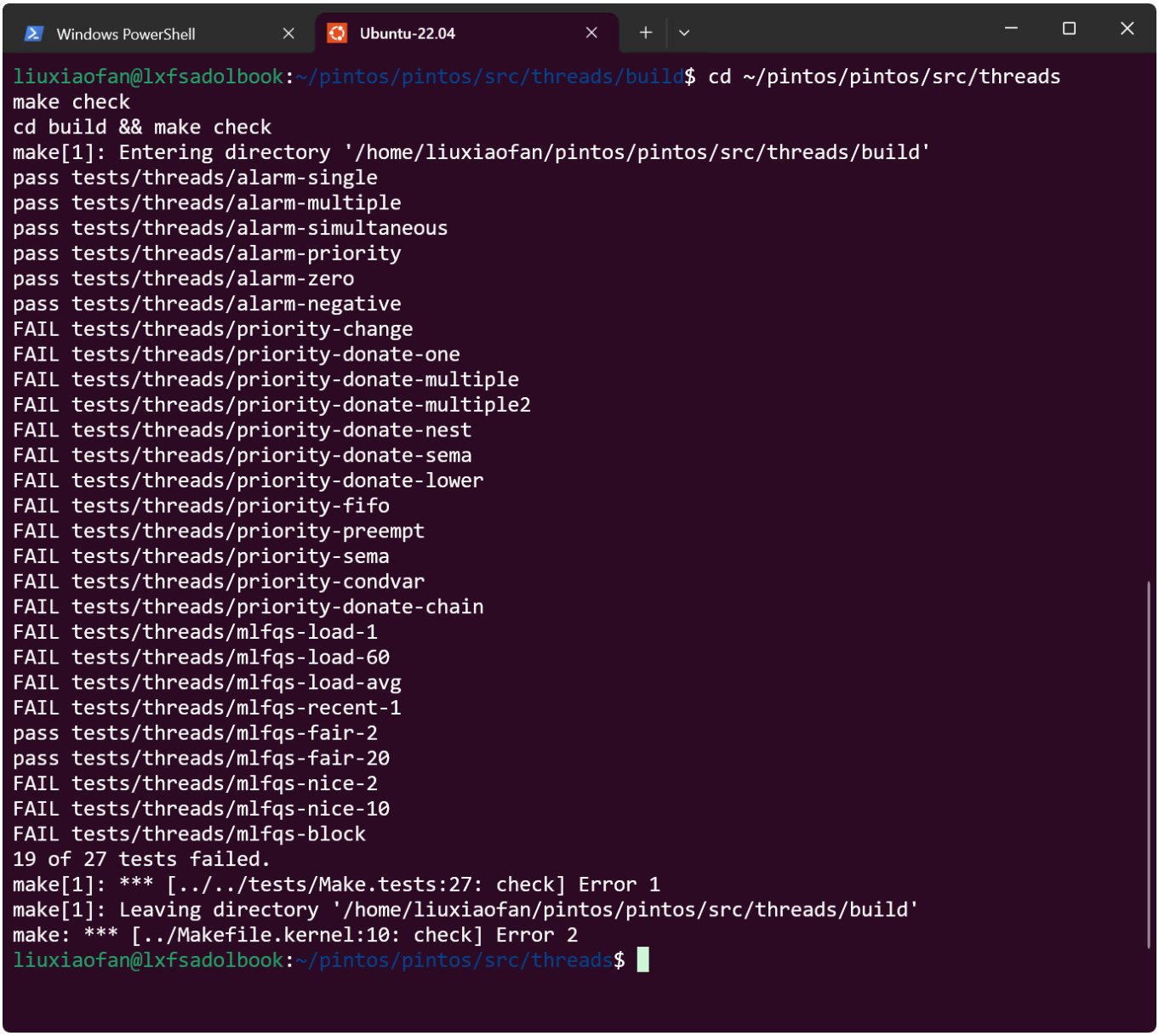
作业 2.3 alarm-priority

通过测试用例的截图：





(1)

alarm-priority 测试的逻辑

测试创建 10 个线程,优先级分别为 `PRI_DEFAULT - (i+5)%10 - 1`,即创建顺序对应优先级 **25,24,23,22,21,30,29,28,27,26**;随后主线程把自身优先级降到 `PRI_MIN`,并 `sema_down` 10 次等待全部子线程完成。所有线程的唤醒时刻相同(`wake_time = timer_ticks() + 5×TIMER_FREQ`, 即 5 秒后):每个线程对齐到 tick 边界后调用 `timer_sleep` 睡到 `wake_time`,醒来打印 `Thread priority N woke up.` 并 `sema_up`。

期望输出(`alarm-priority.ck`)要求唤醒消息**严格按优先级从高到低:30,29,...,21**。其含义是:同一时刻到期的线程，必须按优先级高低依次被调度运行。原实现中 `timer_sleep` 忙等待、就绪队列是 FIFO,到期时谁恰好排在队首先不确定,唤醒顺序随运行相位随机(实测有时 25 开头，偶尔碰巧 30 开头)，无法稳定通过;必须实现**优先级调度**(以及真正的阻塞式睡眠)才能确定性通过。

(2) 需要修改的数据结构和函数

位置	修改
`threads/thread.h`	`struct thread` 增加 `int64_t wake_ticks` 字段(记录唤醒时刻);声明 `thread_sleep` / `thread_wake`
`threads/thread.c`	新增睡眠链表 `sleep_list`(按 wake_ticks 升序)及 `thread_sleep` / `thread_wake` 函数; `thread_unblock` / `thread_yield` 中就绪队列改为**按优先级降序有序插入**(`list_insert_ordered` + 比较函数 `priority_more`),保证队首恒为最高优先级线程
`devices/timer.c`	`timer_sleep` 删除忙等待循环,改为调用 `thread_sleep` 真正阻塞; `timer_interrupt` 每个时钟节拍调用 `thread_wake(ticks)` 唤醒到期线程

(3) 编码实现

`devices/timer.c` —— 阻塞式睡眠 + 时钟中断唤醒:

```
void
timer_sleep (int64_t ticks)
{
  ASSERT (intr_get_level () == INTR_ON);
  if (ticks > 0)
    thread_sleep (timer_ticks () + ticks);    /* 记下目标时刻后阻塞,不再忙等 */
}

static void
timer_interrupt (struct intr_frame *args UNUSED)
{
  ticks++;
  thread_tick ();
  thread_wake (ticks);                        /* 唤醒所有 wake_ticks 已到的线程 */
}
```

`threads/thread.c` —— 睡眠链表与优先级就绪队列:

```
static struct list sleep_list;                /* 睡眠线程,按 wake_ticks 升序 */

/* sleep_list 比较函数:唤醒早者在前 */
static bool sleep_less (a, b, aux UNUSED) { ... wake_ticks 小者为先 ... }
/* ready_list 比较函数:优先级高者在前 */
static bool priority_more (a, b, aux UNUSED) { ... priority 大者为先 ... }

/* 当前线程睡到 wake_ticks 时刻 */
void thread_sleep (int64_t wake_ticks)
```

```
{
  old_level = intr_disable ();
  cur->wake_ticks = wake_ticks;
  list_insert_ordered (&sleep_list, &cur->elem, sleep_less, NULL);
  thread_block ();                              /* 真正阻塞,让出 CPU */
  intr_set_level (old_level);
}

/* 时钟中断中调用:唤醒所有到期线程 */
void thread_wake (int64_t now)
{
  old_level = intr_disable ();
  while (!list_empty (&sleep_list))
    {
      t = list_entry (list_front (&sleep_list), struct thread, elem);
      if (t->wake_ticks > now) break;           /* 链表有序,后面必然未到期 */
      list_remove (&t->elem);
      thread_unblock (t);                       /* 进入按优先级排序的就绪队列 */
    }
  intr_set_level (old_level);
}
```

同时把 `thread_unblock` 和 `thread_yield` 中的 `list_push_back (&ready_list, ...)` 改为 `list_insert_ordered (&ready_list, ..., priority_more, NULL)`。这样多个线程同时到期时,总是优先级最高的线程排在队首先被调度,输出顺序确定为 30→21。

验证结果:重新编译后 `pintos --qemu -- -q run alarm-priority` 输出 `priority 30 woke up.` 严格降序到 `priority 21 woke up.`,与期望输出逐字一致,测试通过;`make check` 中 alarm 系列 6 项全部 pass。统计显示 `490 idle ticks`(原忙等待实现为 0),证明睡眠期间 CPU 真正空闲,效率显著提升。